7章
型や型クラスを自分で作ろう
7.1-5

7.1 - 型を定義する

```
ghci> :i Bool
data Bool = False | True
      型の名前      値コンストラクタ
```

```
data Int = -2147483648|...|-1|0|1|...|2147483648
```

実際にこうなっているわけではないが、意味としてはこうなっている

7.2 - 形づくる

円と長方形を定義するデータ型を作る。

7show.hs

```
data Shape = Circle Float Float Float |
             Rectangle Float Float Float Float
             deriving (Show)

area :: Shape -> Float
area (Circle _ _ r) = pi * r ^ 2
area (Rectangle x1 y1 x2 y2) = (x2 - x1) * (y2 - y1)
```

7.2 - 形づくる

7show.hs

値コンストラクタ　　値コンストラクタに渡す引数

```
data Shape = Circle     Float Float Float |
             Rectangle  Float Float Float Float
             deriving (Show)

area :: Shape -> Float
area (Circle _ _ r) = pi * r ^ 2
area (Rectangle x1 y1 x2 y2) = (x2 - x1) * (y2 - y1)
```

7.2 - 形づくる

7show.hs

```
data Shape = Circle     Float Float Float |
             Rectangle  Float Float Float Float
             deriving (Show)
```

Show型のインスタンスにしてくれている

```
area :: Shape -> Float
area (Circle _ _ r) = pi * r ^ 2
area (Rectangle x1 y1 x2 y2) = (x2 - x1) * (y2 - y1)
```

7.2 - 形づくる

実行例

```
ghci> :t Circle
Circle :: Float -> Float -> Float -> Shape

ghci> area $ Circle 10 20 10
314.15927
ghci> area $ Rectangle 0 0 100 100
10000.0

ghci> Circle 10 20 5
Circle 10.0 20.0 5.0
```

7.2 - 形づくる （課題）

7show.hs

```
data Point = Point Float Float deriving (Show)
data Shape = Circle Point Float |
             Rectangle Point Point deriving (Show)
area :: Shape -> Float
area 円の面積を求める関数を書き換える
area 長方形の面積を求める関数を書き換える
```

実行例

```
ghci> area (Rectangle (Point 0 0) (Point 100 100))
10000.0
ghci> area (Circle (Point 0 0) 24)
1809.5574
```

7.2 - 形づくる （課題）

7show.hs

```
nudge :: Shape -> Float -> Float -> Shape
nudge   円を平行移動させる関数をつくる
nudge   長方形を平行移動させる関数をつくる
```

実行例

```
ghci> nudge (Circle(Point 34 34) 10) 5 10
Circle (Point 39.0 44.0) 10.0
```

7.2 - 形づくる

7show.hs

```
module Shapes (Point(..), Shape(..), area, nudge)where
```

モジュール化することだってできる。

7.2 - 形づくる

7show.hs

```
module Shapes (Point(..), Shape(..), area, nudge)where
```

ある型に含まれている、全ての値コンストラクタ

Point(..)　←同じ→　Point(Point)

Shape(..)　←同じ→　Shape(Circle, Rectangle)

7.2 - 形づくる

```
module Shapes (Point(), Shape(), area, nudge)where
```

データ型はエクスポートできるが
値コンストラクタはエクスポートされない。

→ユーザーは中身をよく知らなくても使える
　作者は内部実装をある程度自由に変更できる

7.3 - レコード構文

```
data Person = Person   String String Int Float String..
```

型の名前　値コンストラクタ　名前　　苗字　　年齢　身長　　などなど

```
firstname :: Person -> String
firstname = (Person firstname _ _ _ ...) = firstname

lasttname :: Person -> String
lastname = (Person _ lastname _ _ _ ...) = lastname
```

.

いっぱい書かないといけないの大変……

.

.

7.3 - レコード構文

7show.hs

```
data Person = Person {firstname :: String,
                      lastname :: String,
                      age :: Int,
                      height :: Float,
                      phoneNumber :: String,
                      flavor :: String} deriving (Show)
```

7.3 - レコード構文

実行例

```
ghci> :t firstname
firstname :: Person -> String

ghci> :t age
age :: Person -> Int
```

なんと、関数を自動的に作成してくれる

7.3 - レコード構文

```
data Car = Car String String Int
```

順番通りにフィールドを埋めないといけない

どのフィールドが何を表しているかわかりにくい

関数を作らないといけない

```
data Car = Car {company :: String,
                model :: String,
                year :: Int} deriving (Show)
```

順番通りに埋めなくてもよい（全て埋まるようにする必要はある）

どのフィールドが何を表しているかわかりやすい

関数が自動的に生成される

7.4 - 型引数

```
data Maybe a = Nothing | Just a
```

型コンストラクタ　型引数　　Nothingじゃない時にこれに保持させたいもの

この型コンストラクタから、Maybe Int , Maybe Char などが作れる

この Maybe に Char を型引数として渡せば、Maybe Char という型が作られる
Haskellには型推論があるため、明示的に型を指定する必要がない

明示したい場合は以下のように

```
ghci> Just 3 :: Maybe Int
Just 3
```

7.4 - 型引数

実行例

```
ghci> Just "Haha"
Just "Haha"
ghci> Just 84
Just 84
ghci> :t Just "Haha"
Just "Haha" :: Maybe String
ghci> :t Just 84
Just 84 :: Num a => Maybe a
ghci> :t Nothing
Nothing :: Maybe a
ghci> Just 10 :: Maybe Double
Just 10.0
```

7.4 - 型引数

型引数は、ある型の中に含まれる、値コンストラクタに複数の型が入る可能性がある場合に非常に有用になる。

```
data Car = Car {company :: String,
                model :: String,
                year :: Int} deriving (Show)
```

```
data Car a b c = Car {company :: a,
                      model :: b,
                      year :: c} deriving (Show)
```

この場合、あまり有用ではない。

7.4 - 型引数 （課題）

7show.hs

```
data Vector a = Vector a a a deriving (Show)

vplus ::(Num a) => Vector a -> Vector a -> Vector a
```

3次元ベクトル同士の加算

```
dotProd :: (Num a) => Vector a -> Vector a -> a
```

3次元ベクトル同士の内積

```
vmult :: (Num a) => Vector a -> a -> Vector a
```

3次元ベクトルのスカラー倍

この場合、3次元ベクトルの計算において、Int型や、Double型など複数の型に対して
実装したいので、型引数の利用が有用になる。

7.4 - 型引数 （課題）

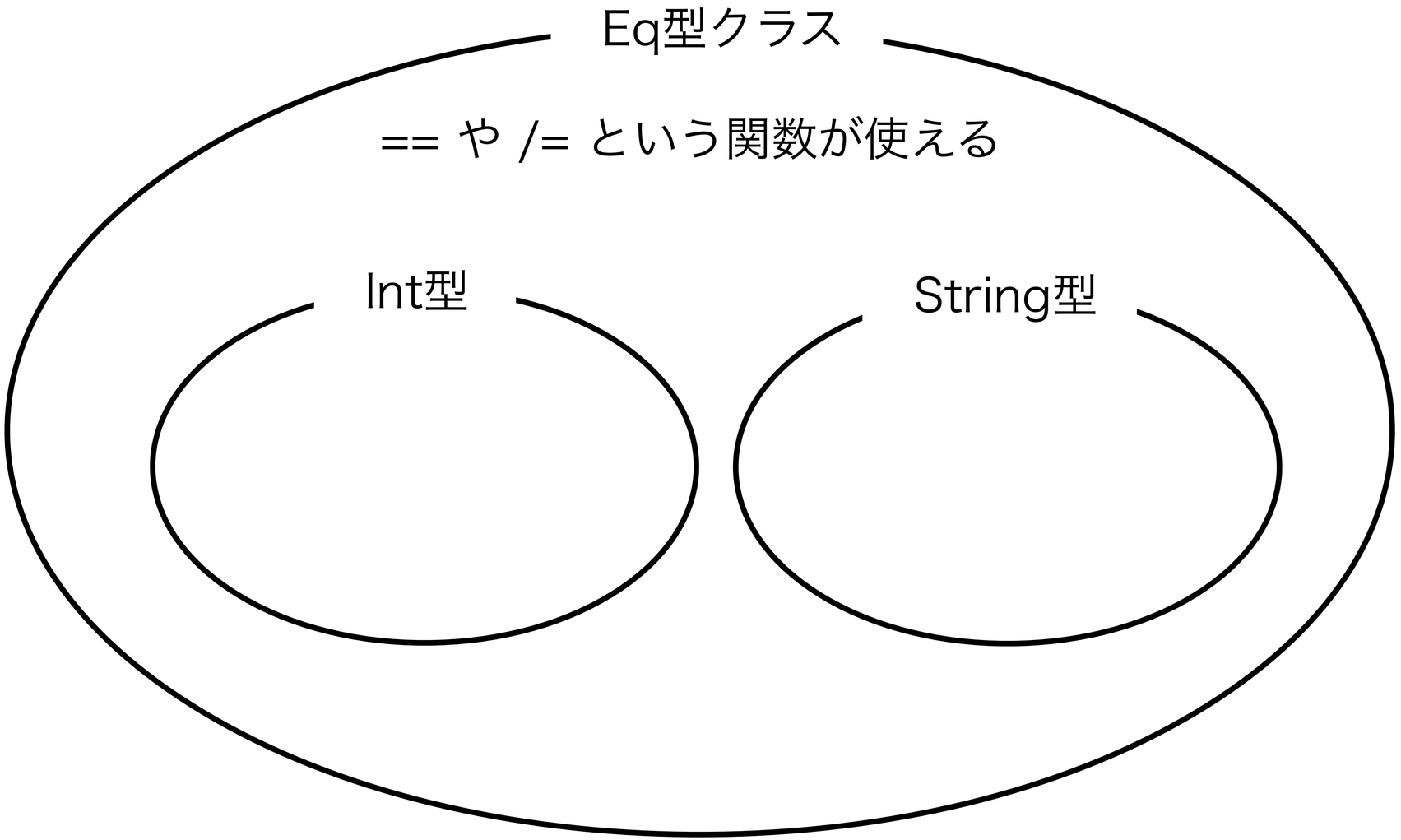
実行例

```
ghci> (Vector 1 2 3) `vplus` (Vector 4 5 6)
Vector 5 7 9

ghci> (Vector 1 2 3) `dotProd` (Vector 4 5 6)
32

ghci> (Vector 1 2 3) `vmult` 4
Vector 4 8 12
```

7.5 - インスタンスの自動導出



7.5 - インスタンスの自動導出

```
data Person = Person {firstname :: String,
                      lastname :: String,
                      age :: Int}
                      deriving (Eq)
```

Int も String もEq型クラスに含まれているため、
これ全体もEq型クラスに含まれていると考えてもよい。

自動導出できる型クラスは、 Eq, Ord, Enum, Bounded, Show, Read

```
data Person = Person {firstname :: String,
                      lastname :: String,
                      age :: Int}
                      deriving (Eq, Show, Read)
```